📘 DOCUMENTO DE ESPECIFICACIÓN MAESTRA Y CÓDIGO BASE (PRD & SDD)

Proyecto: *VeriPago* (SaaS B2B de Prevención de Fraude y Conciliación Instantánea de Transferencias)
Stack Principal: Go (Golang 1.23+) · MySQL 8.0 (InnoDB ACID) · Redis · Google Gemini 2.5 Flash · Mercado Pago API · WhatsApp Cloud API

PARTE 1: VISIÓN DE PRODUCTO Y MODELO DE NEGOCIO

1.1. El Problema Real en el Mercado

En Argentina y LATAM, las transferencias inmediatas reemplazaron al efectivo, generando dos dolores críticos en comercios que venden por WhatsApp o mostrador:

1. **Fraude por comprobantes apócrifos:** Aplicaciones clonadas ("Mercado Pago Fake"), edición de PDFs o reutilización de comprobantes reales de días anteriores ("doble gasto").
2. **Cuello de botella operativo y privacidad:** El dueño del comercio **no quiere ni puede darle acceso a su cuenta de Mercado Pago a sus empleados, cajeros o repartidores** (porque verían el saldo total del negocio o podrían retirar fondos). Hoy, el empleado le reenvía la captura al dueño y espera confirmación manual, demorando entregas o comiéndose estafas en horas pico.

1.2. Propuesta de Valor (La Solución)

Un micro-SaaS B2B donde el titular del comercio vincula su cuenta de Mercado Pago con **un solo clic (OAuth 2.0)** y da de alta los números de WhatsApp de sus vendedores o repartidores. Cuando un cliente envía un comprobante (imagen o PDF), el vendedor **lo reenvía al Bot de WhatsApp del SaaS**. En **menos de 3 segundos**, el sistema:

1. Calcula el hash criptográfico del archivo para descartar duplicados en `0.5 ms`.
2. Analiza forensemente la imagen/PDF con IA Multimodal (extrayendo monto, fecha, ID de operación, COELSA ID, banco origen, alias destino y detectando si es una pantalla de "pago en proceso" en lugar de finalizado).
3. Cruza la información contra la cuenta real de Mercado Pago del comercio mediante Webhooks + API en tiempo real.
4. Bloquea el pago en MySQL mediante transacciones `SELECT ... FOR UPDATE` para que **nunca pueda usarse dos veces**.

5. Responde por WhatsApp con un semáforo claro: ✅ **APROBADO**, ⏳ **EN COLA DE ESPERA (Reintento Automático)** o 🚨 **ALERTA DE FRAUDE / RECHAZado**.

1.3. Modelo de Monetización (Pricing B2B)

Costo operativo estimado por validación (Gemini 2.5 Flash + Infraestructura): < $0.002 USD.

Plan	Precio Mensual (USD / ARS equiv.)	Límite Mensual	Público Objetivo	Funcionalidades Clave
Emprendedor	~$12 USD / mes	300 comprobantes	Vendedores individuales, complejos de canchas chicos	1 cuenta MP, hasta 2 números de WhatsApp autorizados, validación instantánea.
Comercio Pro	~$28 USD / mes	1.500 comprobantes	Gastronomía, pizzerías, heladerías, tiendas	Hasta 2 cuentas MP, 10 empleados/repartidores, alertas de fraude al dueño, exportación Excel/CSV.
Full / Franquicia	~$65 USD / mes	5.000+ comprobantes	Distribuidoras, multimarcas, cadenas	Múltiples cuentas MP, API/Webhooks propios, Generador de Centavos Únicos, pantalla Posnet Virtual.

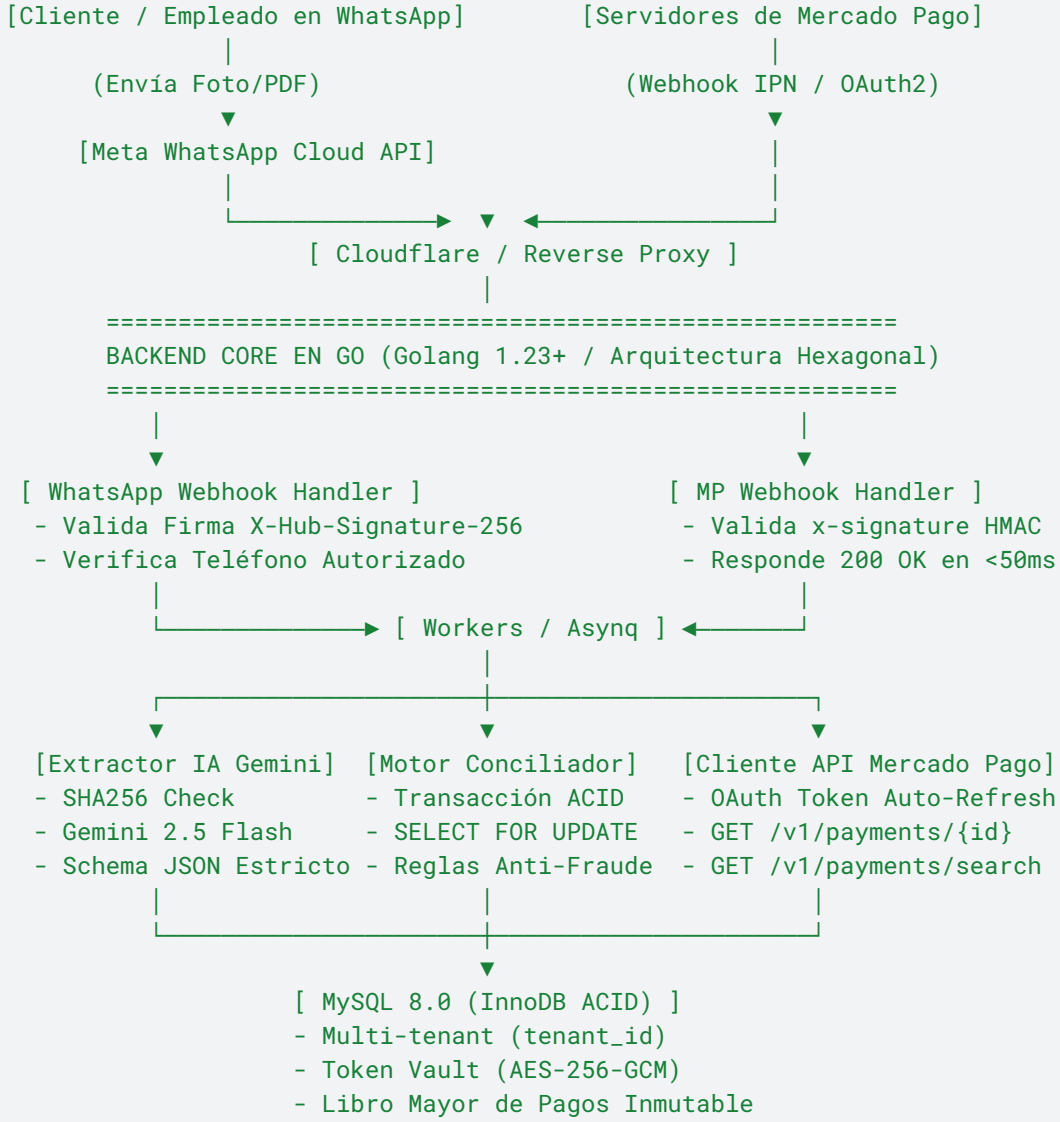
1.4. Innovaciones Clave del Producto

1. **Modo "Reenvío de Empleado" (Onboarding en 2 minutos):** El comercio no cambia su número de WhatsApp ni instala sistemas complejos. El empleado simplemente reenvía la foto que le mandó el cliente al contacto del Bot.
2. **Doble Escudo Anti-Reutilización:** Bloqueo por **Hash SHA-256** del archivo y bloqueo por `mp_payment_id` **consumido** en MySQL.
3. **Cola de Espera Reactiva ("Te aviso apenas entre"):** Si un banco externo demora 40 segundos en acreditar vía COELSA, el bot no rechaza en seco; deja el comprobante en espera en Redis durante 3 minutos y, apenas entra el Webhook de Mercado Pago, le envía un mensaje proactivo al empleado confirmando la acreditación.
4. **Comando `/cobrar` con Centavos/Pesos Críticos:** Para superar la limitación de privacidad de la API de Mercado Pago en transferencias interbancarias (CVU) donde MP no expone el CUIT del emisor, el bot puede asignar centavos o pesos aleatorios (ej. `$10.014`) que actúan como identificador único inconfundible.

PARTE 2: ARQUITECTURA DE SOFTWARE Y SEGURIDAD

2.1. Diagrama de Arquitectura (Monolito Modular Hexagonal en Go)

None



2.2. Estructura de Directorios del Proyecto

None

```
veripago-saas/
├── cmd/
```

```
|   └── api/
|       └── main.go                     # Punto de entrada: Servidor HTTP +
Workers
├── internal/
|   ├── config/
|   |   └── config.go                   # Variables de entorno tipadas
|   ├── domain/
|   |   ├── models.go                   # Entidades de negocio (Tenant, Payment,
Receipt)
|   |   └── ports.go                    # Interfaces para desacoplar
infraestructura
|   ├── application/
|   |   └── verify_receipt_uc.go        # Caso de uso principal de conciliación y
antifraude
|   ├── infrastructure/
|   |   ├── crypto/
|   |   |   └── aes_gcm.go              # Encriptación de Tokens OAuth
(AES-256-GCM)
|   |   ├── ai/
|   |   |   └── gemini_extractor.go     # Integración Gemini 2.5 Flash con JSON
Schema
|   |   ├── mp/
|   |   |   └── client.go               # Cliente HTTP de Mercado Pago + Búsqueda
y OAuth
|   |   └── persistence/
|   |       └── mysql_repo.go           # Repositorio MySQL con transacciones FOR
UPDATE
|   └── interfaces/
|       └── http/
|           ├── webhooks_handler.go     # Endpoints para WhatsApp y Mercado Pago
|           └── middleware_hmac.go      # Validación criptográfica de firmas de
Webhooks
├── migrations/
|   └── 0001_init_schema.sql            # Esquema completo MySQL 8.0
├── docker-compose.yml                  # Entorno local listo para correr
└── go.mod
```

PARTE 3: INFRAESTRUCTURA Y BASE DE DATOS (CÓDIGO COMPLETO)

3.1. `docker-compose.yml`

Archivo listo para levantar MySQL 8.0, Redis y el backend en Go:

```yaml
version: '3.9'

services:
  mysql:
    image: mysql:8.0
    container_name: veripago_mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:-root_secret_local}
      MYSQL_DATABASE: veripago_db
      MYSQL_USER: veripago_user
      MYSQL_PASSWORD: ${DB_PASSWORD:-veripago_pass_local}
      TZ: America/Argentina/Buenos_Aires
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      -
./migrations/0001_init_schema.sql:/docker-entrypoint-initdb.d/0001_ini
t_schema.sql:ro
    command: --default-authentication-plugin=mysql_native_password
--character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci

  redis:
    image: redis:7-alpine
    container_name: veripago_redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

volumes:
  mysql_data:
```

```
  redis_data:
```

3.2. Esquema SQL Transaccional (`migrations/0001_init_schema.sql`)

```sql
CREATE DATABASE IF NOT EXISTS veripago_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE veripago_db;

-- 1. COMERCIOS SUSCRIPTOS (TENANTS)
CREATE TABLE tenants (
    id CHAR(36) PRIMARY KEY,
    business_name VARCHAR(150) NOT NULL,
    cuit VARCHAR(13) UNIQUE NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    plan_type ENUM('FREE_TRIAL', 'EMPRENDEDOR', 'PRO', 'FULL') DEFAULT
'FREE_TRIAL',
    monthly_quota INT NOT NULL DEFAULT 50,
    used_quota INT NOT NULL DEFAULT 0,
    time_window_minutes INT NOT NULL DEFAULT 120,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 2. CUENTAS VINCULADAS DE MERCADO PAGO (TOKENS ENCRIPTADOS CON AES-256-GCM)
CREATE TABLE mp_accounts (
    id CHAR(36) PRIMARY KEY,
    tenant_id CHAR(36) NOT NULL,
    mp_user_id BIGINT NOT NULL,
    expected_alias VARCHAR(100) NULL,
    expected_cvu VARCHAR(22) NULL,
    expected_holder_name VARCHAR(150) NULL,
    access_token_enc TEXT NOT NULL,
    refresh_token_enc TEXT NOT NULL,
    token_expires_at TIMESTAMP NOT NULL,
    webhook_secret VARCHAR(255) NULL,
    status ENUM('CONNECTED', 'EXPIRED', 'REVOKED') DEFAULT 'CONNECTED',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE KEY uk_mp_user (mp_user_id),
    INDEX idx_tenant_mp (tenant_id),
```

```
    CONSTRAINT fk_mp_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON
DELETE CASCADE
) ENGINE=InnoDB;

-- 3. NÚMEROS DE WHATSAPP DE EMPLEADOS / VENDEDORES AUTORIZADOS
CREATE TABLE authorized_senders (
    id CHAR(36) PRIMARY KEY,
    tenant_id CHAR(36) NOT NULL,
    phone_number VARCHAR(25) NOT NULL,
    employee_name VARCHAR(100) NOT NULL,
    role ENUM('OWNER', 'CASHIER', 'DELIVERY') DEFAULT 'CASHIER',
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uk_phone_tenant (phone_number, tenant_id),
    INDEX idx_phone_lookup (phone_number),
    CONSTRAINT fk_sender_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON
DELETE CASCADE
) ENGINE=InnoDB;

-- 4. LIBRO MAYOR DE PAGOS REALES DE MERCADO PAGO (FUENTE DE LA VERDAD)
CREATE TABLE mp_payments (
    id CHAR(36) PRIMARY KEY,
    tenant_id CHAR(36) NOT NULL,
    mp_payment_id BIGINT NOT NULL,
    amount DECIMAL(15, 2) NOT NULL,
    currency VARCHAR(5) DEFAULT 'ARS',
    mp_status VARCHAR(30) NOT NULL,
    operation_type VARCHAR(50) NULL,
    payer_cuit VARCHAR(20) NULL,
    payer_name VARCHAR(150) NULL,
    coelsa_id VARCHAR(60) NULL,
    payment_date TIMESTAMP NOT NULL,
    reconciliation_status ENUM('AVAILABLE', 'CONSUMED', 'DISPUTED') DEFAULT
'AVAILABLE',
    consumed_at TIMESTAMP NULL,
    consumed_by_receipt_id CHAR(36) NULL,
    raw_json JSON NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uk_tenant_mppayment (tenant_id, mp_payment_id),
    INDEX idx_matching_search (tenant_id, reconciliation_status, amount,
payment_date)
) ENGINE=InnoDB;

-- 5. AUDITORÍA DE COMPROBANTES PROCESADOS
CREATE TABLE receipt_validations (
    id CHAR(36) PRIMARY KEY,
    tenant_id CHAR(36) NOT NULL,
    sender_phone VARCHAR(25) NOT NULL,
```

```
    wa_message_id VARCHAR(120) UNIQUE NOT NULL,
    file_sha256 CHAR(64) NOT NULL,
    extracted_amount DECIMAL(15, 2) NULL,
    extracted_op_number VARCHAR(100) NULL,
    extracted_coelsa_id VARCHAR(100) NULL,
    extracted_date TIMESTAMP NULL,
    extracted_sender_name VARCHAR(150) NULL,
    extracted_sender_cuit VARCHAR(20) NULL,
    extracted_receiver_alias_or_cbu VARCHAR(150) NULL,
    extracted_source_bank VARCHAR(100) NULL,
    ai_fraud_warnings JSON NULL,
    validation_result ENUM(
        'VERIFIED_EXACT_ID',
        'VERIFIED_HEURISTIC',
        'REJECTED_NOT_FOUND',
        'REJECTED_WRONG_DESTINATION',
        'FRAUD_ALREADY_USED',
        'FRAUD_DUPLICATE_FILE',
        'FRAUD_INCOMPLETE_TRANSFER',
        'INVALID_RECEIPT_IMAGE'
    ) NOT NULL,
    matched_mp_payment_id CHAR(36) NULL,
    processing_time_ms INT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_tenant_sha (tenant_id, file_sha256),
    INDEX idx_tenant_created (tenant_id, created_at),
    CONSTRAINT fk_val_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    CONSTRAINT fk_val_mppayment FOREIGN KEY (matched_mp_payment_id) REFERENCES
mp_payments(id)
) ENGINE=InnoDB;
```

PARTE 4: IMPLEMENTACIÓN EN GO (CÓDIGO CORE DEL MVP)

4.1. Modelos de Dominio e Interfaces (`internal/domain/models.go`)

```
Go
package domain

import "time"

type ValidationStatus string
```

```
const (
	StatusVerifiedExactID      ValidationStatus = "VERIFIED_EXACT_ID"
	StatusVerifiedHeuristic    ValidationStatus = "VERIFIED_HEURISTIC"
	StatusRejectedNotFound     ValidationStatus = "REJECTED_NOT_FOUND"
	StatusRejectedWrongDest    ValidationStatus = "REJECTED_WRONG_DESTINATION"
	StatusFraudAlreadyUsed     ValidationStatus = "FRAUD_ALREADY_USED"
	StatusFraudDuplicateFile   ValidationStatus = "FRAUD_DUPLICATE_FILE"
	StatusFraudIncomplete      ValidationStatus = "FRAUD_INCOMPLETE_TRANSFER"
	StatusInvalidReceiptImage  ValidationStatus = "INVALID_RECEIPT_IMAGE"
)

// ExtractedReceiptData representa la salida estructurada que devuelve Gemini 2.5
Flash
type ExtractedReceiptData struct {
	IsValidReceipt          bool      `json:"is_valid_receipt"`
	IsCompletedTransfer     bool      `json:"is_completed_transfer"` // False si
dice "Programada", "En proceso" o "¿Confirmás el envío?"
	Amount                  float64   `json:"amount"`
	Currency                string    `json:"currency"`
	OperationNumber         string    `json:"operation_number"`          // ID
de Mercado Pago o Nro de Comprobante bancario
	CoelsaID                string    `json:"coelsa_id"`                 //
Código COELSA de 22 caracteres si figura
	SourceBankOrWallet      string    `json:"source_bank_or_wallet"`     // Ej:
"Mercado Pago", "Brubank", "Banco Galicia", "MODO"
	SenderName              string    `json:"sender_name"`
	SenderCUIT              string    `json:"sender_cuit"`               // Solo
dígitos
	ReceiverName            string    `json:"receiver_name"`
	ReceiverAliasOrCVU      string    `json:"receiver_alias_or_cvu"`
	ReceiptTimestampISO     string    `json:"receipt_timestamp_iso"`
	VisualManipulationNotes []string  `json:"visual_manipulation_notes"` //
Alertas si hay fuentes desalineadas o artefactos
}

// MPPayment representa un ingreso real acreditado en Mercado Pago
type MPPayment struct {
	ID                   string
	TenantID             string
	MPPaymentID          int64
	Amount               float64
	Currency             string
	MPStatus             string
	OperationType        string
	PayerCUIT            string
	PayerName            string
	CoelsaID             string
	PaymentDate          time.Time
```

```
	ReconciliationStatus string // "AVAILABLE" | "CONSUMED"
	RawJSON              []byte
}

// VerificationOutcome es la respuesta final que se formatea hacia WhatsApp
type VerificationOutcome struct {
	Status          ValidationStatus
	MessageForUser  string
	MatchedPayment  *MPPayment
	ExtractedData   *ExtractedReceiptData
	ProcessingTime  time.Duration
}
```

4.2. Seguridad: Encriptación AES-256-GCM para Tokens OAuth (`internal/infrastructure/crypto/aes_gcm.go`)

Nunca guardamos el `access_token` de Mercado Pago de los comercios en texto plano en MySQL:

```
Go
package crypto

import (
	"crypto/aes"
	"crypto/cipher"
	"crypto/rand"
	"encoding/base64"
	"errors"
	"io"
)

type TokenVault struct {
	masterKey []byte // Debe ser exactamente de 32 bytes (AES-256)
}

func NewTokenVault(masterKey32Bytes string) (*TokenVault, error) {
	key := []byte(masterKey32Bytes)
	if len(key) != 32 {
		return nil, errors.New("MASTER_KEY debe tener exactamente 32 bytes
para AES-256-GCM")
	}
	return &TokenVault{masterKey: key}, nil
}
```

```
func (v *TokenVault) Encrypt(plainText string) (string, error) {
	block, err := aes.NewCipher(v.masterKey)
	if err != nil {
		return "", err
	}

	gcm, err := cipher.NewGCM(block)
	if err != nil {
		return "", err
	}

	nonce := make([]byte, gcm.NonceSize())
	if _, err := io.ReadFull(rand.Reader, nonce); err != nil {
		return "", err
	}

	cipherText := gcm.Seal(nonce, nonce, []byte(plainText), nil)
	return base64.StdEncoding.EncodeToString(cipherText), nil
}

func (v *TokenVault) Decrypt(encodedCipherText string) (string, error) {
	data, err := base64.StdEncoding.DecodeString(encodedCipherText)
	if err != nil {
		return "", err
	}

	block, err := aes.NewCipher(v.masterKey)
	if err != nil {
		return "", err
	}

	gcm, err := cipher.NewGCM(block)
	if err != nil {
		return "", err
	}

	nonceSize := gcm.NonceSize()
	if len(data) < nonceSize {
		return "", errors.New("ciphertext demasiado corto")
	}

	nonce, cipherText := data[:nonceSize], data[nonceSize:]
	plainBytes, err := gcm.Open(nil, nonce, cipherText, nil)
	if err != nil {
		return "", err
	}

	return string(plainBytes), nil
```

```
}
```

4.3. Extractor Forense con Gemini 2.5 Flash (`internal/infrastructure/ai/gemini_extractor.go`)

Utiliza el SDK oficial `google.golang.org/genai` forzando un esquema JSON estricto (`ResponseSchema`) entrenado en las particularidades de comprobantes bancarios argentinos:

```
Go
package ai

import (
	"context"
	"encoding/json"
	"fmt"

	"google.golang.org/genai"
	"veripago-saas/internal/domain"
)

type GeminiReceiptExtractor struct {
	client *genai.Client
	model  string
}

func NewGeminiReceiptExtractor(ctx context.Context, apiKey string)
(*GeminiReceiptExtractor, error) {
	client, err := genai.NewClient(ctx, &genai.ClientConfig{
		APIKey:  apiKey,
		Backend: genai.BackendGeminiAPI,
	})
	if err != nil {
		return nil, err
	}
	return &GeminiReceiptExtractor{
		client: client,
		model:  "gemini-2.5-flash",
	}, nil
}

const forensicSystemPrompt = `Eres un perito informático antifraude especializado
en comprobantes de transferencia bancaria y billeteras virtuales de Argentina
```

```
(Mercado Pago, Brubank, Ualá, Naranja X, MODO, Santander, Galicia, BBVA, Banco
Nación, Cuenta DNI, Macro, etc.).
Tu tarea es extraer con precisión quirúrgica los datos del comprobante adjunto y
detectar intentos de estafa.

REGLAS CRÍTICAS DE EXTRACCIÓN Y DETECCIÓN DE FRAUDE:
1. is_completed_transfer: Ponlo en FALSE si la imagen es una pantalla previa de
confirmación (ej: "¿Confirmás la transferencia?", botón "Transferir" abajo), o si
dice "Transferencia programada", "En proceso", "Pendiente" o "Rechazada". Solo es
TRUE si indica claramente que el envío ya se realizó.
2. monto (amount): Extrae el número decimal exacto. En Argentina el punto (.)
suele separar miles y la coma (,) o superíndice los centavos (ej: "$ 15.450,50" ->
15450.50).
3. operation_number: Si es un comprobante de Mercado Pago, extrae el "Número de
operación" (suele tener 10 a 12 dígitos numéricos). Si es de otro banco, extrae el
número de comprobante u operación principal.
4. coelsa_id: Si aparece el "Código de identificación COELSA" o "ID COELSA" (suele
tener 22 caracteres alfanuméricos empezando con números), extráelo completo.
5. sender_cuit: Extrae el CUIT/CUIL de quien envía el dinero solo con dígitos (sin
guiones).
6. receiver_alias_or_cvu: Extrae el Alias, CBU o CVU del destinatario para
verificar que le hayan transferido al comercio correcto.
7. visual_manipulation_notes: Si notas tipografías de distinto tamaño o fuente en
el monto respecto del resto del texto, recortes sospechosos o bordes desalineados
típicos de edición o apps falsas ("MP Fake"), descríbelo en este array.`

func (g *GeminiReceiptExtractor) ExtractFromMedia(ctx context.Context, fileBytes
[]byte, mimeType string) (*domain.ExtractedReceiptData, error) {
	schema := &genai.Schema{
		Type: genai.TypeObject,
		Properties: map[string]*genai.Schema{
			"is_valid_receipt":        {Type: genai.TypeBoolean},
			"is_completed_transfer":   {Type: genai.TypeBoolean},
			"amount":                  {Type: genai.TypeNumber},
			"currency":                {Type: genai.TypeString},
			"operation_number":        {Type: genai.TypeString},
			"coelsa_id":               {Type: genai.TypeString},
			"source_bank_or_wallet":   {Type: genai.TypeString},
			"sender_name":             {Type: genai.TypeString},
			"sender_cuit":             {Type: genai.TypeString},
			"receiver_name":           {Type: genai.TypeString},
			"receiver_alias_or_cvu":   {Type: genai.TypeString},
			"receipt_timestamp_iso":   {Type: genai.TypeString},
			"visual_manipulation_notes": {
				Type:  genai.TypeArray,
				Items: &genai.Schema{Type: genai.TypeString},
			},
		},
```

```
            Required: []string{
                  "is_valid_receipt", "is_completed_transfer", "amount",
                  "operation_number", "source_bank_or_wallet",
"receiver_alias_or_cvu",
            },
      }

      temp := float32(0.0)
      config := &genai.GenerateContentConfig{
            SystemInstruction: &genai.Content{
                  Parts: []*genai.Part{{Text: forensicSystemPrompt}},
            },
            Temperature:      &temp,
            ResponseMIMEType: "application/json",
            ResponseSchema:   schema,
      }

      contents := []*genai.Content{
            {
                  Role: "user",
                  Parts: []*genai.Part{
                        {InlineData: &genai.Blob{Data: fileBytes, MIMEType:
mimeType}},
                        {Text: "Analiza este comprobante y devuelve el JSON
estructurado."},
                  },
            },
      }

      resp, err := g.client.Models.GenerateContent(ctx, g.model, contents, config)
      if err != nil {
            return nil, fmt.Errorf("error llamando a Gemini API: %w", err)
      }

      var extracted domain.ExtractedReceiptData
      if err := json.Unmarshal([]byte(resp.Text()), &extracted); err != nil {
            return nil, fmt.Errorf("error parseando JSON de Gemini: %w", err)
      }

      return &extracted, nil
}
```

4.4. Cliente de Mercado Pago (`internal/infrastructure/mp/client.go`)

Permite consultar un pago puntual por ID o sincronizar los últimos ingresos por CVU con `/v1/payments/search`:

```go
package mp

import (
	"context"
	"encoding/json"
	"fmt"
	"io"
	"net/http"
	"net/url"
	"time"

	"veripago-saas/internal/domain"
)

type MPClient struct {
	httpClient *http.Client
}

func NewMPClient() *MPClient {
	return &MPClient{
		httpClient: &http.Client{Timeout: 8 * time.Second},
	}
}

type mpPaymentResponse struct {
	ID              int64   `json:"id"`
	Status          string  `json:"status"`
	OperationType   string  `json:"operation_type"`
	TransactionAmt  float64 `json:"transaction_amount"`
	CurrencyID      string  `json:"currency_id"`
	DateCreated     string  `json:"date_created"`
	DateApproved    string  `json:"date_approved"`
	Payer           struct {
		FirstName      string `json:"first_name"`
		LastName       string `json:"last_name"`
		Identification struct {
			Type   string `json:"type"`
			Number string `json:"number"`
		} `json:"identification"`
	} `json:"payer"`
}
```

```
// GetPaymentByID busca directamente un número de operación en Mercado Pago
func (c *MPClient) GetPaymentByID(ctx context.Context, accessToken string,
tenantID string, paymentID string) (*domain.MPPayment, error) {
	endpoint := fmt.Sprintf("https://api.mercadopago.com/v1/payments/%s",
url.PathEscape(paymentID))
	req, err := http.NewRequestWithContext(ctx, http.MethodGet, endpoint, nil)
	if err != nil {
		return nil, err
	}
	req.Header.Set("Authorization", "Bearer "+accessToken)

	resp, err := c.httpClient.Do(req)
	if err != nil {
		return nil, err
	}
	defer resp.Body.Close()

	if resp.StatusCode == http.StatusNotFound {
		return nil, nil // No existe ese pago en la cuenta del comercio
	}
	if resp.StatusCode != http.StatusOK {
		return nil, fmt.Errorf("MP API devolvió status %d", resp.StatusCode)
	}

	bodyBytes, err := io.ReadAll(resp.Body)
	if err != nil {
		return nil, err
	}

	return mapRawMPToDomain(tenantID, bodyBytes)
}

// FetchRecentApprovedPayments trae las últimas transferencias acreditadas para
alimentar la tabla local
func (c *MPClient) FetchRecentApprovedPayments(ctx context.Context, accessToken
string, tenantID string) ([]*domain.MPPayment, error) {
	u, _ := url.Parse("https://api.mercadopago.com/v1/payments/search")
	q := u.Query()
	q.Set("status", "approved")
	q.Set("sort", "date_created")
	q.Set("criteria", "desc")
	q.Set("limit", "30")
	q.Set("range", "date_created")
	q.Set("begin_date", "NOW-12HOURS")
	q.Set("end_date", "NOW")
	u.RawQuery = q.Encode()
```

```
	req, err := http.NewRequestWithContext(ctx, http.MethodGet, u.String(), nil)
	if err != nil {
		return nil, err
	}
	req.Header.Set("Authorization", "Bearer "+accessToken)

	resp, err := c.httpClient.Do(req)
	if err != nil {
		return nil, err
	}
	defer resp.Body.Close()

	if resp.StatusCode != http.StatusOK {
		return nil, fmt.Errorf("MP Search API devolvió status %d",
resp.StatusCode)
	}

	var searchResult struct {
		Results []json.RawMessage `json:"results"`
	}
	if err := json.NewDecoder(resp.Body).Decode(&searchResult); err != nil {
		return nil, err
	}

	var payments []*domain.MPPayment
	for _, raw := range searchResult.Results {
		p, err := mapRawMPToDomain(tenantID, raw)
		if err == nil && p != nil {
			payments = append(payments, p)
		}
	}
	return payments, nil
}

func mapRawMPToDomain(tenantID string, rawJSON []byte) (*domain.MPPayment, error)
{
	var parsed mpPaymentResponse
	if err := json.Unmarshal(rawJSON, &parsed); err != nil {
		return nil, err
	}

	payDate, err := time.Parse(time.RFC3339, parsed.DateApproved)
	if err != nil {
		payDate, _ = time.Parse(time.RFC3339, parsed.DateCreated)
	}

	return &domain.MPPayment{
		TenantID:            tenantID,
```

```
            MPPaymentID:          parsed.ID,
            Amount:               parsed.TransactionAmt,
            Currency:             parsed.CurrencyID,
            MPStatus:             parsed.Status,
            OperationType:        parsed.OperationType,
            PayerCUIT:            parsed.Payer.Identification.Number,
            PayerName:            fmt.Sprintf("%s %s", parsed.Payer.FirstName,
parsed.Payer.LastName),
            PaymentDate:          payDate,
            ReconciliationStatus: "AVAILABLE",
            RawJSON:              rawJSON,
      }, nil
}
```

4.5. Repositorio MySQL con Bloqueo Pesimista (`SELECT ... FOR UPDATE`) (`internal/infrastructure/persistence/mysql_repo.go`)

Garantiza que si dos sucursales o dos empleados intentan validar el mismo pago al mismo milisegundo, MySQL serializa el acceso y solo el primero lo consume:

```
Go
package persistence

import (
      "context"
      "database/sql"
      "errors"
      "time"

      "github.com/google/uuid"
      "veripago-saas/internal/domain"
)

var (
      ErrPaymentAlreadyConsumed = errors.New("el pago ya fue consumido previamente
por otro comprobante")
      ErrPaymentNotFound        = errors.New("no se encontró un pago acreditado
disponible")
)

type MySQLRepository struct {
      db *sql.DB
```

```
}

func NewMySQLRepository(db *sql.DB) *MySQLRepository {
	return &MySQLRepository{db: db}
}

// CheckFileDuplicate verifica en <1ms si el mismo archivo exacto ya fue enviado
antes
func (r *MySQLRepository) CheckFileDuplicate(ctx context.Context, tenantID string,
fileSHA256 string) (bool, time.Time, error) {
	var createdAt time.Time
	query := `SELECT created_at FROM receipt_validations WHERE tenant_id = ? AND
file_sha256 = ? LIMIT 1`
	err := r.db.QueryRowContext(ctx, query, tenantID,
fileSHA256).Scan(&createdAt)
	if errors.Is(err, sql.ErrNoRows) {
		return false, time.Time{}, nil
	}
	if err != nil {
		return false, time.Time{}, err
	}
	return true, createdAt, nil
}

// UpsertMPPayment inserta un pago proveniente de Webhook o de la API de MP sin
pisar su estado si ya fue consumido
func (r *MySQLRepository) UpsertMPPayment(ctx context.Context, p
*domain.MPPayment) error {
	if p.ID == "" {
		p.ID = uuid.NewString()
	}
	query := `
		INSERT INTO mp_payments (
			id, tenant_id, mp_payment_id, amount, currency, mp_status,
			operation_type, payer_cuit, payer_name, coelsa_id,
payment_date,
			reconciliation_status, raw_json
		) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 'AVAILABLE', ?)
		ON DUPLICATE KEY UPDATE
			mp_status = VALUES(mp_status),
			payer_cuit = COALESCE(NULLIF(VALUES(payer_cuit), ''),
payer_cuit)
	`
	_, err := r.db.ExecContext(ctx, query,
		p.ID, p.TenantID, p.MPPaymentID, p.Amount, p.Currency, p.MPStatus,
		p.OperationType, p.PayerCUIT, p.PayerName, p.CoelsaID, p.PaymentDate,
string(p.RawJSON),
	)
```

```
	return err
}

// ClaimPaymentByExactID bloquea la fila con FOR UPDATE, verifica que no haya sido
usada y que coincida el monto, y la marca como CONSUMED
func (r *MySQLRepository) ClaimPaymentByExactID(ctx context.Context, tenantID
string, mpPaymentID int64, expectedAmount float64, receiptValidationID string)
(*domain.MPPayment, error) {
	tx, err := r.db.BeginTx(ctx, &sql.TxOptions{Isolation:
sql.LevelReadCommitted})
	if err != nil {
		return nil, err
	}
	defer tx.Rollback()

	var p domain.MPPayment
	querySelect := `
		SELECT id, mp_payment_id, amount, mp_status, payment_date,
reconciliation_status
		FROM mp_payments
		WHERE tenant_id = ? AND mp_payment_id = ?
		FOR UPDATE
	`
	err = tx.QueryRowContext(ctx, querySelect, tenantID, mpPaymentID).Scan(
		&p.ID, &p.MPPaymentID, &p.Amount, &p.MPStatus, &p.PaymentDate,
&p.ReconciliationStatus,
	)
	if errors.Is(err, sql.ErrNoRows) {
		return nil, ErrPaymentNotFound
	}
	if err != nil {
		return nil, err
	}

	if p.ReconciliationStatus == "CONSUMED" {
		return &p, ErrPaymentAlreadyConsumed
	}

	// Verificamos que el pago esté aprobado y el monto coincida (con tolerancia
de 0.01 por redondeo)
	if p.MPStatus != "approved" || absDiff(p.Amount, expectedAmount) > 0.01 {
		return nil, ErrPaymentNotFound
	}

	queryUpdate := `
		UPDATE mp_payments
		SET reconciliation_status = 'CONSUMED', consumed_at = NOW(),
consumed_by_receipt_id = ?
```

```
			WHERE id = ?
		`
		if _, err := tx.ExecContext(ctx, queryUpdate, receiptValidationID, p.ID);
err != nil {
			return nil, err
		}

		if err := tx.Commit(); err != nil {
			return nil, err
		}
		return &p, nil
}

// ClaimPaymentByHeuristic (Para transferencias desde otros bancos por CVU)
// Busca un pago AVAILABLE con el monto exacto dentro de la ventana de tiempo
permitida por el comercio
func (r *MySQLRepository) ClaimPaymentByHeuristic(ctx context.Context, tenantID
string, expectedAmount float64, timeWindowMinutes int, receiptValidationID string)
(*domain.MPPayment, error) {
		tx, err := r.db.BeginTx(ctx, &sql.TxOptions{Isolation:
sql.LevelReadCommitted})
		if err != nil {
			return nil, err
		}
		defer tx.Rollback()

		var p domain.MPPayment
		querySelect := `
			SELECT id, mp_payment_id, amount, mp_status, payment_date,
reconciliation_status
			FROM mp_payments
			WHERE tenant_id = ?
			  AND reconciliation_status = 'AVAILABLE'
			  AND mp_status = 'approved'
			  AND amount = ?
			  AND payment_date >= DATE_SUB(NOW(), INTERVAL ? MINUTE)
			ORDER BY payment_date DESC
			LIMIT 1
			FOR UPDATE
		`
		err = tx.QueryRowContext(ctx, querySelect, tenantID, expectedAmount,
timeWindowMinutes).Scan(
			&p.ID, &p.MPPaymentID, &p.Amount, &p.MPStatus, &p.PaymentDate,
&p.ReconciliationStatus,
		)
		if errors.Is(err, sql.ErrNoRows) {
			return nil, ErrPaymentNotFound
		}
```

```
	if err != nil {
		return nil, err
	}

	queryUpdate := `
		UPDATE mp_payments
		SET reconciliation_status = 'CONSUMED', consumed_at = NOW(),
consumed_by_receipt_id = ?
		WHERE id = ?
	`
	if _, err := tx.ExecContext(ctx, queryUpdate, receiptValidationID, p.ID);
err != nil {
		return nil, err
	}

	if err := tx.Commit(); err != nil {
		return nil, err
	}
	return &p, nil
}

func absDiff(a, b float64) float64 {
	if a > b {
		return a - b
	}
	return b - a
}
```

4.6. Caso de Uso Orquestador (`internal/application/verify_receipt_uc.go`)

Une la validación SHA-256, el análisis de Gemini Vision, la consulta a Mercado Pago y el bloqueo en MySQL, generando el mensaje listo para responder en WhatsApp:

```
Go
package application

import (
	"context"
	"crypto/sha256"
	"encoding/hex"
	"errors"
	"fmt"
	"regexp"
```

```
	"strconv"
	"strings"
	"time"

	"github.com/google/uuid"
	"veripago-saas/internal/domain"
	"veripago-saas/internal/infrastructure/ai"
	"veripago-saas/internal/infrastructure/mp"
	"veripago-saas/internal/infrastructure/persistence"
)

var numericRegex = regexp.MustCompile(`^\d{10,13}$`)

type VerifyReceiptUseCase struct {
	repo      *persistence.MySQLRepository
	extractor *ai.GeminiReceiptExtractor
	mpClient  *mp.MPClient
}

func NewVerifyReceiptUseCase(
	repo *persistence.MySQLRepository,
	extractor *ai.GeminiReceiptExtractor,
	mpClient *mp.MPClient,
) *VerifyReceiptUseCase {
	return &VerifyReceiptUseCase{
		repo:      repo,
		extractor: extractor,
		mpClient:  mpClient,
	}
}

func (uc *VerifyReceiptUseCase) Execute(
	ctx context.Context,
	tenantID string,
	mpAccessToken string,
	expectedAliasOrCVU string,
	timeWindowMinutes int,
	fileBytes []byte,
	mimeType string,
) (*domain.VerificationOutcome, error) {
	start := time.Now()
	validationID := uuid.NewString()

	// 1. ESCUDO 1: Hash SHA-256 del archivo (Costo $0, 1ms)
	hash := sha256.Sum256(fileBytes)
	fileSHA := hex.EncodeToString(hash[:])

	isDup, prevTime, err := uc.repo.CheckFileDuplicate(ctx, tenantID, fileSHA)
```

```
	if err != nil {
		return nil, err
	}
	if isDup {
		return &domain.VerificationOutcome{
			Status: domain.StatusFraudDuplicateFile,
			MessageForUser: fmt.Sprintf(
				"🚨 *ALERTA DE FRAUDE (ARCHIVO REPETIDO)*\n\nEsta misma
imagen/PDF ya fue enviada y procesada el *%s*.\n❌ *NO ENTREGAR EL PEDIDO.*",
				prevTime.Format("02/01/2006 a las 15:04 hs"),
			),
			ProcessingTime: time.Since(start),
		}, nil
	}

	// 2. ESCUDO 2: Extracción Forense con Gemini 2.5 Flash
	extracted, err := uc.extractor.ExtractFromMedia(ctx, fileBytes, mimeType)
	if err != nil {
		return nil, err
	}

	if !extracted.IsValidReceipt {
		return &domain.VerificationOutcome{
			Status:         domain.StatusInvalidReceiptImage,
			MessageForUser: "⚠️ *Imagen no válida*\nNo pudimos detectar un
comprobante de transferencia legible en la foto enviada.",
			ExtractedData:  extracted,
			ProcessingTime: time.Since(start),
		}, nil
	}

	if !extracted.IsCompletedTransfer {
		return &domain.VerificationOutcome{
			Status: domain.StatusFraudIncomplete,
			MessageForUser: "🚨 *CUIDADO: TRANSFERENCIA NO
FINALIZADA*\n\nLa captura muestra una transferencia *programada, en proceso o en
la pantalla previa a confirmar*.\n❌ *El dinero NO fue enviado.*",
			ExtractedData:  extracted,
			ProcessingTime: time.Since(start),
		}, nil
	}

	// 3. ESCUDO 3: Verificar que el destinatario del comprobante sea el
comercio (Si el comprobante muestra Alias/CVU)
	if expectedAliasOrCVU != "" && extracted.ReceiverAliasOrCVU != "" {
		cleanExpected :=
strings.ToLower(strings.TrimSpace(expectedAliasOrCVU))
```

```
            cleanExtracted :=
strings.ToLower(strings.TrimSpace(extracted.ReceiverAliasOrCVU))
            if !strings.Contains(cleanExtracted, cleanExpected) &&
!strings.Contains(cleanExpected, cleanExtracted) {
                  return &domain.VerificationOutcome{
                        Status: domain.StatusRejectedWrongDest,
                        MessageForUser: fmt.Sprintf(
                              "❌ *DESTINATARIO INCORRECTO*\n\nEl comprobante
indica que se transfirió a *%s*, que no coincide con tu cuenta configurada
(*%s*).",
                              extracted.ReceiverAliasOrCVU, expectedAliasOrCVU,
                        ),
                        ExtractedData:  extracted,
                        ProcessingTime: time.Since(start),
                  }, nil
            }
      }

      // 4. ESCUDO 4: Conciliación contra Mercado Pago API + Bloqueo Transaccional
en MySQL
      cleanOpNumber := strings.TrimSpace(extracted.OperationNumber)

      // 4.A: Si el número de operación parece un ID directo de Mercado Pago (10 a
13 dígitos)
      if numericRegex.MatchString(cleanOpNumber) {
            mpIDInt, _ := strconv.ParseInt(cleanOpNumber, 10, 64)

            // Consultamos a la API de MP por si el Webhook aún no llegó y
actualizamos la tabla local
            remotePayment, err := uc.mpClient.GetPaymentByID(ctx, mpAccessToken,
tenantID, cleanOpNumber)
            if err == nil && remotePayment != nil {
                  _ = uc.repo.UpsertMPPayment(ctx, remotePayment)
            }

            claimedPayment, err := uc.repo.ClaimPaymentByExactID(ctx, tenantID,
mpIDInt, extracted.Amount, validationID)
            if errors.Is(err, persistence.ErrPaymentAlreadyConsumed) {
                  return &domain.VerificationOutcome{
                        Status: domain.StatusFraudAlreadyUsed,
                        MessageForUser: fmt.Sprintf(
                              "🚨 *INTENTO DE DOBLE USO DETECTADO*\n\nLa
operación *#%d* por *$%.2f* es real, pero *YA FUE UTILIZADA* para validar otro
pedido anteriormente.\n❌ *NO ENTREGAR.*",
                              mpIDInt, extracted.Amount,
                        ),
                        ExtractedData:  extracted,
                        ProcessingTime: time.Since(start),
```

```
                }, nil
        }
        if err == nil && claimedPayment != nil {
                return &domain.VerificationOutcome{
                        Status:         domain.StatusVerifiedExactID,
                        MatchedPayment: claimedPayment,
                        ExtractedData:  extracted,
                        MessageForUser: fmt.Sprintf(
                                "✅ *TRANSFERENCIA VERIFICADA (100%% ID
Exacto)*\n\n💰 *Monto acreditado:* $%.2f\n🔢 *ID Mercado Pago:* #%d\n🏦 *Origen:*
%s\n🕒 *Hora acreditación:* %s\n🔒 *Estado:* Bloqueado y asignado a esta venta.",
                                claimedPayment.Amount,
claimedPayment.MPPaymentID, extracted.SourceBankOrWallet,
claimedPayment.PaymentDate.Format("15:04:05 hs"),
                        ),
                        ProcessingTime: time.Since(start),
                }, nil
        }
    }

    // 4.B: Si viene desde otro Banco / Billetera (CVU), sincronizamos los
últimos pagos y buscamos por Monto + Ventana de Tiempo
    recentPayments, err := uc.mpClient.FetchRecentApprovedPayments(ctx,
mpAccessToken, tenantID)
    if err == nil {
        for _, p := range recentPayments {
            _ = uc.repo.UpsertMPPayment(ctx, p)
        }
    }

    claimedHeuristic, err := uc.repo.ClaimPaymentByHeuristic(ctx, tenantID,
extracted.Amount, timeWindowMinutes, validationID)
    if err == nil && claimedHeuristic != nil {
        return &domain.VerificationOutcome{
            Status:         domain.StatusVerifiedHeuristic,
            MatchedPayment: claimedHeuristic,
            ExtractedData:  extracted,
            MessageForUser: fmt.Sprintf(
                "✅ *TRANSFERENCIA BANCARIA ACREDITADA*\n\n💰 *Monto:*
$%.2f\n🏦 *Banco origen (Comprobante):* %s\n👤 *Emisor en comprobante:* %s\n🔢 *ID
interno MP:* #%d\n🕒 *Impactó a las:* %s\n🔒 *Estado:* Pago consumido y
asignado.",
                claimedHeuristic.Amount, extracted.SourceBankOrWallet,
extracted.SenderName, claimedHeuristic.MPPaymentID,
claimedHeuristic.PaymentDate.Format("15:04:05 hs"),
            ),
            ProcessingTime: time.Since(start),
        }, nil
```

```
	}

	// 5. Si no se encontró en Mercado Pago
	return &domain.VerificationOutcome{
		Status:        domain.StatusRejectedNotFound,
		ExtractedData: extracted,
		MessageForUser: fmt.Sprintf(
			"❌ *NO IMPACTÓ EN MERCADO PAGO*\n\nNo se encontró ningún
ingreso disponible por *$%.2f* en tu cuenta de Mercado Pago.\n\n• *Origen
declarado:* %s\n• *Operación:* %s\n⏳ _Si el cliente acaba de transferir desde
otro banco, te avisaremos automáticamente si impacta en los próximos 3 minutos._",
			extracted.Amount, extracted.SourceBankOrWallet,
extracted.OperationNumber,
		),
		ProcessingTime: time.Since(start),
	}, nil
}
```

4.7. Middleware de Validación Criptográfica de Webhooks de Mercado Pago (`internal/interfaces/http/middleware_hmac.go`)

Impide que un atacante envíe peticiones `POST` falsas a tu servidor simulando ser Mercado Pago:

```Go
package http

import (
	"crypto/hmac"
	"crypto/sha256"
	"encoding/hex"
	"fmt"
	"net/http"
	"strings"
)

// ValidateMPWebhookSignature verifica la cabecera x-signature oficial de Mercado
Pago
func ValidateMPWebhookSignature(webhookSecret string, r *http.Request) bool {
	xSignature := r.Header.Get("x-signature")
	xRequestID := r.Header.Get("x-request-id")
	dataID := r.URL.Query().Get("data.id")

	if xSignature == "" || xRequestID == "" || dataID == "" {
```

```
		return false
	}

	var ts, v1Hash string
	parts := strings.Split(xSignature, ",")
	for _, part := range parts {
		kv := strings.SplitN(strings.TrimSpace(part), "=", 2)
		if len(kv) == 2 {
			if kv[0] == "ts" {
				ts = kv[1]
			} else if kv[0] == "v1" {
				v1Hash = kv[1]
			}
		}
	}

	if ts == "" || v1Hash == "" {
		return false
	}

	// Template oficial exigido por Mercado Pago Developers:
	manifest := fmt.Sprintf("id:%s;request-id:%s;ts:%s;", dataID, xRequestID,
ts)

	mac := hmac.New(sha256.New, []byte(webhookSecret))
	mac.Write([]byte(manifest))
	expectedHash := hex.EncodeToString(mac.Sum(nil))

	return hmac.Equal([]byte(expectedHash), []byte(v1Hash))
}
```

PARTE 5: GUÍA DE PUESTA EN MARCHA RÁPIDA (COMANDOS)

Para inicializar tu repositorio local en Go con este código:

Shell

```
# 1. Crear carpeta e inicializar el módulo en Go
mkdir veripago-saas && cd veripago-saas
go mod init veripago-saas

# 2. Instalar las dependencias oficiales
go get google.golang.org/genai
```

```
go get github.com/go-sql-driver/mysql
go get github.com/google/uuid

# 3. Levantar la infraestructura local (MySQL 8.0 + Redis)
docker compose up -d
```